Domaći zadatak 9.2

Datum: 10.11.2026.



Zadatak

1.Kreirati klasu BankAccount koja predstavlja bankovni račun. Klasa treba da ima sledeće atribute:

- owner – ime vlasnika računa;
- balance – trenutno stanje na računu.

Implementirati sledeće metode:

1. deposit(amount: float) -> BankAccount – uvećava stanje na računu za zadati iznos. Ako je iznos pozitivan, izvršava uplatu. Vraća objekat nad kojim je metoda pozvana.
2. withdraw(amount: float) -> BankAccount – umanjuje stanje na računu za zadati iznos. Isplata se izvršava samo ako je iznos pozitivan i ako na računu ima dovoljno novca. Vraća objekat nad kojim je metoda pozvana.
3. display_balance() -> None – ispisuje trenutno stanje na računu.

Kreirati objekat klase BankAccount i omogućiti izvršavanje sledećeg niza poziva:

account = BankAccount("Ana", 10000)
account.deposit(5000).withdraw(2000).deposit(1000)
account.display_balance()

Očekivani izlaz:
Stanje na racunu: 14000

Domaći zadatak 9.2

Datum: 10.11.2026.



Zadatak

2.Kreirati klasu UserProfile koja predstavlja profil korisnika.
Klasa treba da ima sledeće atribute:

- username – korisničko ime;
- email – imejl adresa;
- city – grad;
- active – informacija o tome da li je profil aktivan.

Prilikom kreiranja objekta, profil treba da bude aktivan.

Implementirati sledeće metode:

1. change_email(email: str) -> UserProfile – menja imejl adresu korisnika i vraća objekat.
2. change_city(city: str) -> UserProfile – menja grad korisnika i vraća objekat.
3. deactivate() -> UserProfile – deaktivira profil i vraća objekat.
4. activate() -> UserProfile – aktivira profil i vraća objekat.
5. display_info() -> None – ispisuje sve informacije o korisniku.

Testirati metode tako da se mogu povezati u jedan izraz:

```
user = UserProfile("ana123", "ana@gmail.com", "Nis")
user.change_email("ana@yahoo.com").change_city("Beograd").d
eactivate()
user.display_info()
```

Domaći zadatak 9.2

Datum: 10.11.2026.



Zadatak

3.Kreirati klasu Product koja predstavlja proizvod u prodavnici. Klasa treba da ima sledeće atribute:

- name – naziv proizvoda;
- price – cena proizvoda;
- quantity – količina proizvoda na stanju.

Implementirati sledeće metode:

1. change_price(new_price: float) -> Product – menja cenu proizvoda ako je nova cena pozitivna. Vraća objekat.
2. add_stock(amount: int) -> Product – povećava količinu proizvoda na stanju za zadati iznos ako je iznos pozitivan. Vraća objekat.
3. sell(amount: int) -> Product – umanjuje količinu proizvoda na stanju ako ima dovoljno proizvoda i ako je količina pozitivna. Vraća objekat.
4. display_info() -> None – ispisuje informacije o proizvodu.

Kreirati objekat klase Product i omogućiti sledeći način korišćenja:

```
product = Product("Laptop", 80000, 10)
product.change_price(75000).sell(2).add_stock(5).sell(3)
product.display_info()
```